From Idea to KDP

Writing a Book with a Crew of AI Editors

Jacob Verhoeks

2026

From Idea to KDP

Writing a Book with a Crew of AI Editors

Jacob Verhoeks

Copyright

The commands, scripts, and file templates printed in this book may be used freely in your own projects. They are also a working repository, `ai-book-tutorial`, which contains the project scaffold, the review-crew definitions, the cover and preflight scripts, and the full review archive for this book. The author publishes its location alongside this edition; if you are holding a printed copy and cannot find it, every file it contains is small enough to type from these pages.

A note on the code in this book: the page is 5.5 inches wide, which fits 55 characters of monospaced type. Longer commands are wrapped with a trailing backslash and still run as printed. Longer lines of program output are wrapped with the continuation indented, and where output is abridged the text says so.

Every command in this book was executed on the machine that produced it. Prices, royalty rates, spine calculations, and dashboard wording belong to Amazon KDP and change without notice; the book names the authority for each one and tells you to check it. Nothing here is legal, tax, or financial advice.

This book was produced with the informed, decisive use of AI assistance, including drafting and editorial review, under the author's direction. The afterword states exactly what the machine did, what it got wrong, and what the author decided. The disclosure answers given to Amazon KDP match this page.

Dedication

For everyone who has a book in a folder called `notes`.

Part One: Before You Write Anything

Four chapters, three files, and no prose. Everything in this part is cheap to change, which is the only reason it comes first.

Chapter 1: A Book Is a Chain of Documents

> The one-sentence model: premise, voice, outline, chapters, review, revision, publish, each document worthless until the one before it is right. Where AI is strong and where it is dangerous. The separation rule. What this book is not. Name the files the reader owns by the end of part one.

I wrote three novels in 2026 with AI as the drafting engine and a rotating crew of AI editors as the review board. The first took four months of thrashing. The third took six weeks of calm, with the same tools and the same model. What changed was the order.

The rest is context. The first book shipped at 95,000 words, the second went through eight full drafts, and the third was built from a story bible before a single sentence of prose existed. Total spend across all three was less than a human editor charges for one manuscript review, which is the number people repeat back to me and the least interesting thing about the project.

A book is not one act of writing. It is a chain of documents, and each document is worthless until the one before it is right:

Read that as a dependency graph, because that is what it is. The outline cannot be judged without the premise, since a scene is only wrong relative to what the book is for. The chapters cannot be judged without the voice guide, since prose is only off-key relative to a key. The review cannot be judged without the outline, since pacing is a comparison between what a chapter promised and what it delivered. Skip a document and you do not save its cost. You move that cost into a later stage where it is multiplied by the number of pages already written.

Where the machine is strong

Three jobs, and it is genuinely good at all three.

Volume with constraints. Give a model a beat, a voice spec, and a canon file, and it will produce 2,800 words of on-target prose in a minute. Not finished prose. On-target prose, which is a different and more useful thing, because finishing is editing and editing comes later.

Tireless review. A human editor reads your manuscript once and gets tired around chapter twelve. Six agents read it six times, in parallel, each one looking at exactly one dimension, at four in the morning, for the price of a sandwich. This is the part of the pipeline that has no non-AI equivalent at any price I can afford.

Counting. How many times does the manuscript say “almost”? Do the dates in chapter nine

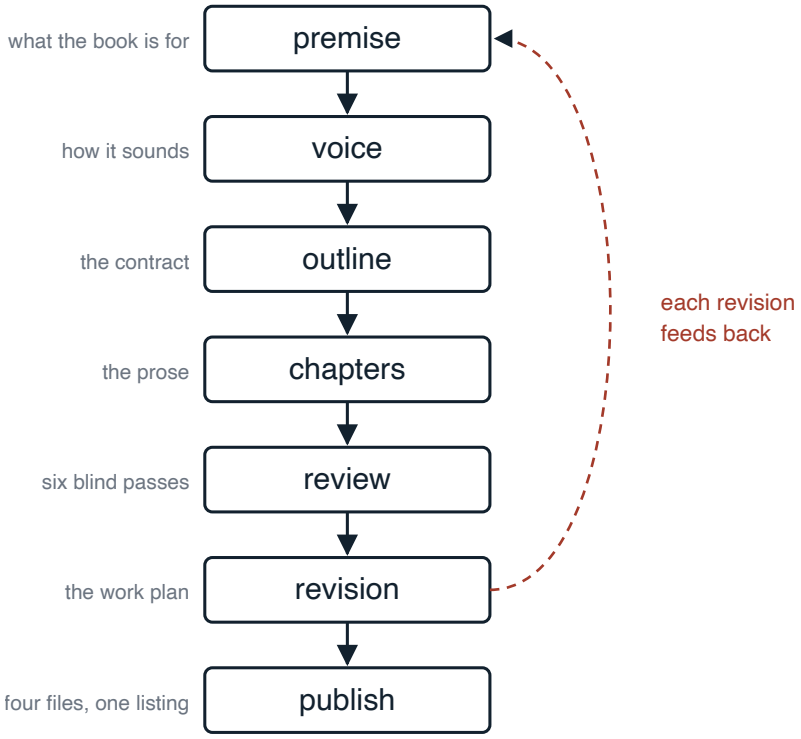

Figure 1: The pipeline: seven documents, in order, with every revision feeding back to the premise.

agree with chapter three? Is that phrase repeated in two chapters forty pages apart? A model with a shell is very good at this, and a human is very bad at it.

Where the machine is dangerous

Two jobs, and it will happily do both if you let it.

Deciding what the book is. Ask a model for a premise and you get a competent average of every premise in its training data. That is exactly the book nobody takes home. The premise is your job. The model's job is to interrogate it.

Judging its own prose. A model asked "is this good?" will tell you yes, in detail, with reasons. A model asked "count every sentence that violates rule four of this style spec, with line numbers" will tell you something true. The difference is not the model's intelligence, it is whether the question has a checkable answer.

Out of that split falls the one rule the rest of this book is built on: separate the writing from the editing. You draft with one process, one voice, one file open. You review with a crew of specialists, each of which checks one thing and never touches the prose. Mixing the two produces mush at both ends. The prose gets "improved" into a blender, and the reviews turn into taste, which cannot be acted on.

What this book is

It is the full background behind a blog post. The post gives you the pipeline in twenty minutes. This book gives you the reasoning, the failure modes, the files, and the second half that the post compresses into three paragraphs: everything that happens between "the manuscript is done" and "the paperback is on sale".

It is also specific about its own evidence. Every command printed here was run on the machine that wrote the book. Where a command misbehaved, the misbehaviour is in the text, including the one in chapter 6 where the project's own word count counted 16 words where 7 existed. Where a fact belongs to Amazon rather than to me, the book names the page you check instead of quoting a number that will be stale before the paperback ships.

What this book is not

Not a prompt pack. There are four prompts in it, one each for interrogating a premise, extracting a voice, drafting a chapter and running the review orbit, and they are long and boring because they are specifications rather than magic words.

Not a marketing book. It covers the listing: the description, the categories, the keywords, the price, the disclosure. It stops there. Advertising, newsletters, and launch teams are a different craft and I have not earned the right to teach them.

Not a case for unattended AI writing. The pipeline actively prevents that, and the prevention is the product. By the end you will have a voice guide extracted from your own writing, a review record showing every finding and your response to it, and a git history where each decision has a name on it. That record is the evidence that the book is yours, and the day someone asks you the question, you will want it.

Not a fiction book or a non-fiction book. The pipeline was built on novels, and this book is non-fiction produced by the same pipeline, so both paths are described where they diverge, in the chapter where they diverge.

What you own by the end of part one

Three files, none of them prose, in a folder that will still be there in five years:

- `bible/book-premise.md`, which is what the book is for and the three cliches it is forbidden to drift into.
- `bible/reader-and-promise.md`, which is who buys it and what they were promised.
- `bible/voice.md`, which is the technical spec for prose that reads like a person wrote it.

None of those three is writing. All three are decisions.

The decision this chapter closes

Whether to work in this order at all. If you are going to draft first and decide what the book is later, close the book here and keep the fifteen minutes.

If you are going to work in pipeline order, make the folder before you turn the page, and put one file in it: `bible/` with a `book-premise.md` containing nothing but your three candidate ideas, one line each. The next chapter kills two of them.

Chapter 2: The Idea, and the Premise File That Kills Bad Ideas

> An idea is not a premise. The six sections of the premise file and why each is a filter rather than a description. The interrogation method: the author answers, the AI asks. Worked example on a novel one-liner, then the same six sections on this book's own premise.

Most people arrive with an idea that sounds like this: "an AI thriller, but a realistic one." Or: "a book about writing books with AI." Both are topics. A topic is a direction of travel, and a book needs a destination.

The difference is testable. A premise can be wrong. A topic cannot, which is why it feels so comfortable to hold one for three years.

So the first file you write is a filter, not a description. Six sections, and each one is designed to fail your idea while failing is still free.

The six sections

One sentence. The whole book: who, what pressure, what stakes. Not a plot summary. The sentence that makes a stranger take the book home. Mine for the third novel:

> Twelve days before a Patagonian clearinghouse will settle the largest AI-generated zero-day auction yet, a Buenos Aires SRE discovers that its supposedly independent witnesses have begun certifying the future.

Count what is doing work in there. A clock: twelve days. A specific machine: a clearinghouse that settles auctions for exploits. A specific wrongness: witnesses certifying things that have not happened. And a protagonist with boots on the ground rather than a job title from a movie. Take any one of those four out and the sentence goes soft.

What makes this different. My shelf already holds fifty AI thrillers. So does yours. Naming the genre is not an answer. Name the machine and the market that the other fifty do not have. If you cannot, the honest conclusion is that you have a topic and someone else has already written the book.

The door-in. What ordinary skill walks your protagonist into the extraordinary situation? An SRE who maintains the boring half of security finds one blade that should be cold and is warm. A platform engineer who reads clocks notices a timestamp that cannot exist. The door-in is what

makes competence plausible, and competence is what makes a reader trust the next 300 pages. In non-fiction the same question applies to the reader: what do they already know how to do, and how does the book enter through that?

The central question. One question that only the whole book answers. If a chapter answers it, the book is a short story with padding. If nothing in the book answers it, you have written a mood.

The real-world anchor. What exists today that the reader recognises, so the fiction feels true before it feels exciting? For non-fiction this is the evidence base: the three novels, the public template, and KDP's own forms, in my case.

Cliche guardrails. Name the three most obvious ways this draft could drift into a book that already exists, and write down the rule in this file that forbids each. This section is the one everyone skips and the one that pays. A draft drifts toward cliche by accident, chapter by chapter, and you will not notice from inside. A named cliche with a named rule is a thing a review agent can check mechanically in month three.

The interrogation method

Here is the part that inverts what most people do with AI. You do not ask the model for a premise. You give it your answers and instruct it to attack them.

> I am writing the premise file for a book. I will answer six questions and you will interrogate the answers. For each one: name the weakest word in my answer, name the book on the shelf my answer is already describing, and ask the one question I am avoiding. Do not write my premise. Do not offer alternatives unless I ask.

The last two sentences matter more than the rest. Without them the model writes your premise for you, in the voice of the average of all premises, and it will be smooth, and you will accept it because it is smoother than yours. Then you spend four months writing a book you did not choose. I know how long it takes because I did it on book one.

Run the interrogation on three ideas at once. It is the cheapest comparison you will ever get, and two of the three will die in about an hour. Let them. An idea that cannot survive six questions from a model that has read everything will not survive 80,000 words and a stranger's twenty minutes in a bookshop.

The same six sections, non-fiction

This book went through its own filter, and the file is in the project as `bible/book-premise.md`. Its one-sentence answer:

> A working engineer's account of taking a book from a first idea to a live Amazon KDP listing, using plain text, Pandoc, and a crew of AI editors that review each other's blind spots instead of writing the book for you.

The "what makes this different" section is where the interrogation drew blood. My first answer was "a practical guide to writing with AI", which describes roughly four hundred books published

in the last two years. The question I was avoiding: what do you have that a prompt-list author does not? The answer that survived: the actual files, run against three finished books and against this book itself, with the failures left in. That answer then changed the book. It is why chapter 6 contains a bug report about my own Makefile, and why the afterword is a ledger rather than a thank-you note.

The cliche guardrails did the same job in the other direction. Three named drifts: the tool tour, AI triumphalism, and the productivity-guru voice. Each one has a rule in the file, and each rule is checkable. No command appears without the decision that needed it. Every claim about what AI did well is paired with the specific thing it got wrong. No word-count challenges, no morning routines, no author brand.

The decision this chapter closes

Write the file. Six sections, your answers, in your words, with the interrogation transcript thrown away and only the conclusions kept. Then stress it: name the three cliches and the three rules.

You will notice the file is short, maybe 500 words, and took two hours. That ratio is correct and it repeats through part one. These are the cheapest words in the project, and every later stage cites them.

When `bible/book-premise.md` exists and two of your three ideas are dead, you are done here.

Chapter 3: Who Buys This, Asked Early

> The commercial question belongs at the start. Reader, promise, the back-cover claim, the shelf test, and for non-fiction the prior-knowledge lists.

Most writing advice puts the audience question at the end, in a chapter called marketing, next to a paragraph about social media. That ordering is why so many finished manuscripts cannot be sold. By the time you ask who this is for, the book has already answered, and it has answered "me".

Asking early costs one file and about ninety minutes. It changes what you write in chapter fourteen.

The file

`bible/reader-and-promise.md`. In a novel's bible this slot holds the cast list. For non-fiction the cast is the reader, and the file holds five things.

Primary reader. Not a demographic. A person with a situation. Mine:

> A software or infrastructure engineer, 30-55, who writes for work and wants to write a book. Has a terminal, git, an AI subscription, and opinions about text editors. Does not have an agent, a publisher, an ISBN, or any idea what a spine width is. Their fear is producing something that reads generated, and being found out.

The fear line is the most useful sentence in the file. It told me the book needed a voice chapter before the tooling chapters, and it is why the review section is four chapters long instead of one.

Secondary reader, and what the prose owes them. Mine is a working writer who is AI-curious and file-comfortable. What they are owed: no paragraph that requires reading a Makefile to understand. Every command explained in prose first. That single line has constrained every page since.

Who this is not for. Say it in the book, early, without apology. My chapter 1 tells the prompt-pack reader they are in the wrong place. Losing that reader on page 3 is a gift to both of us. A reader who feels lied to writes a one-star review that is entirely deserved.

The promise, in the reader's words. First person, their voice, one sentence:

> "By the end I will know exactly which file to open next, at every stage from the first idea to a live listing, and I will have a way to prove the writing is mine."

Every chapter of this book is checkable against that sentence. Two draft chapters died against it.

The back-cover claim the promise supports. Yes, a year early. It will change, and drafting it now is still the fastest structural test available, because a promise you cannot compress into three lines is a promise the book has not made yet.

The shelf test beats the genre label

"Contemporary techno thriller" is a label. It tells you nothing about whether someone buys the book.

The shelf test: name three books a bookseller would put yours next to, then say in one sentence what a reader gets from yours that they do not get from those three. If the answer is "mine is better", you have not done the test. If the answer is "those three are about the crime and mine is about the plumbing that made the crime possible", you have a jacket, a category, and probably your keywords.

For non-fiction, name the three books and the two blog posts. Blog posts are competitors in non-fiction and pretending otherwise is how you write a 200-page version of something the reader already read for free. My shelf test answer: those books teach prompting, and this one ships the build system, the review crew, and the publishing half.

The two lists that only non-fiction needs

At the bottom of the file, two lists. What the prose may assume the reader knows, and what it may not.

Mine, may assume: command line, git basics, Markdown, what a PDF is, what an ebook is.

May not assume: Pandoc, LaTeX, typography, trim sizes, bleed, KDP's dashboard, royalty models, ISBN mechanics, how AI subagents get spawned.

That second list is the most-consulted file in this project. It is why chapter 15 explains bleed in a sentence before it explains the cover script, and why nothing here explains what a commit is. Without the lists you write for a reader whose knowledge drifts to match whatever you learned most recently, which reads to everyone else as a book that keeps changing its mind about who it is talking to.

For fiction the equivalent question is what the reader is assumed to know about the world before chapter one, and the discipline is the same: write it down, or the draft will decide for you, inconsistently.

The decision this chapter closes

Write the file. Five sections and the two lists. Then do one more thing, which takes five minutes and saves a year: send the promise sentence and the back-cover claim to two people who match the primary reader, and ask one question. Not "does this sound good". Ask: "would you have bought this, and what did you think it was about?"

Their answer to the second half is your real blurb. Keep it in the file.

When `bible/reader-and-promise.md` exists, the commercial question is closed until chapter 14, where a commercial pass reopens it with the finished book in hand. The next file is the one that

decides whether the prose sounds like you.

Chapter 4: Your Voice, Extracted Before the Machine Writes a Word

> The step everyone skips. Build the corpus from your own best writing, run the extraction prompt, treat the output as a technical spec. The em-dash rule and the 22 exceptions in my own corpus. What to do with no corpus. The read-aloud test.

This is the chapter that decides whether your book reads like a person wrote it.

It is also the step every AI writing guide leaves out, and I understand why: it produces no prose, it takes an afternoon, and it feels like homework. Skip it and you will spend that afternoon anyway, later, rewriting chapter after chapter by instinct, wondering why the whole thing sounds like a competent stranger.

The voice guide is a technical spec. Not a mood board, not a list of favourite authors. A file a review agent can check a paragraph against, mechanically, and report violations with line numbers.

Build the corpus first

Collect your own best writing. Not everything you have written: your best, because the guide extracted from your average will describe your average.

My corpus was 26 published articles from 2022 to 2026, plus the bible files from the three novels. Yours might be work documents, incident write-ups, long emails, a decade of blog posts. Volume matters less than honesty. Ten pieces where you sounded like yourself beat fifty where you sounded like your employer.

Put the corpus in a folder. Then hand it over with an instruction that produces a spec rather than a compliment.

The extraction prompt

> You are analysing writing samples to extract a reusable voice guide. Read every sample. Produce `bible/voice.md` containing: 1. **Who is writing.** The persona, in one paragraph. 2. **Sentence patterns.** Quote three short sentences from the samples that carry the weight of a paragraph, and show the long-then-short cadence this author uses. 3. **Paragraph length.** Typical and maximum, and the rule for when a single-sentence paragraph is allowed. 4. **Directness.** A table: instead of X, this author writes Y. Use real examples from the samples. 5. **Transparency about failure.** How dead ends and mis-

> takes get stated. 6. **Humour.** When it lands and when it is cut. 7. **Forbidden words and patterns.** Exact phrases this author never uses. 8. **The test.** One line that tells me later whether a paragraph is in this voice. Write the guide in the extracted voice itself.

That last instruction is a check on the whole exercise. If the guide reads like a style manual, the extraction failed and you run it again on a better corpus. If the guide sounds like you, it worked.

The two sections that do the work

The **directness table** and the **forbidden list** carry more weight than the other six sections combined, because they are the two a machine can enforce.

The forbidden list is where AI-slop vocabulary goes to die. Mine bans delve, dive into, unlock, unleash, elevate, supercharge, game-changer, seamless, robust as praise, leverage as a verb, journey, landscape, realm, tapestry, testament to, it's worth noting, at the end of the day, and about fifteen more. It also bans three patterns, which are harder to see and worse: rhetorical questions used as transitions, sentences that announce their own structure, and encouragement.

The directness table is the positive half. One row from mine:

Instead of	This author writes
The tool may occasionally produce errors	Three of my review findings were wrong, one factually dangerous

That is not a word swap. It is a rule about specificity, expressed as an example, which is the only form of style instruction a model reliably follows.

The em-dash rule, and my own 22 violations

The highest-return mechanical edit available:

> **No em-dashes (U+2014). Ever.**

The em-dash is the loudest machine tell in current AI prose. Removing it from a generated draft does more for perceived humanity than any amount of rewriting, because the reader's pattern-matcher is calibrated on exactly that character. Split the sentence. Use a comma, a colon, or parentheses.

Now the honest part. The corpus this book's voice guide was extracted from contains 22 em-dashes. I checked:

```
grep -o '—' blog/write-a-book-with-agent-editors.md \
  | wc -l
# 22
```

The blog predates the rule. The rule applies to book prose from the voice file forward, and the book's own review pass counts em-dashes in `chapters/` and reports zero in prose, with three inside

code fences where the character is the thing being searched for. Chapter 9 shows that grep and that distinction. That is the shape of every rule in this pipeline: written down, dated, and verified by a command rather than by a promise.

Machine tells are a density problem

The other tells are rationed rather than banned, because banning them is what produces stilted prose. Your voice guide needs the list and the ration; it does not need the method. The watch list, the counting method and the structural fix are in chapter 9, where you will be holding a draft to run them against.

What belongs in the spec is the ration itself. For each tell, write the rate you will tolerate and the unit you will measure it in, and write it before you have any prose to be defensive about. Mine says at most one contrast structure per chapter, and a blind review pass later found three chapters over that line, which is exactly the argument for writing the number down while it costs nothing.

If you have no corpus

Then you have three hours of work instead of one.

Write 2,000 words of something true and unpublished: how a project failed, what a place smells like at 6am, why you left a job. Do not write it with AI. Then extract the guide from that, plus a list of tells you personally hate in other people's writing, plus your own hard lines.

Do not extract a voice guide from authors you admire. You will get their voice, badly, and every review pass afterwards will enforce someone else's rhythm on your book.

The test at the bottom of the file

Read a paragraph aloud. If it could have been written by any AI or any senior engineer with a content calendar, it is not your voice yet. If it sounds like someone who has stared at this specific problem for three days and finally figured it out, that is the voice, and that is the one line you will use a hundred times.

The decision this chapter closes

Whose rules the prose obeys. When `bible/voice.md` exists and sounds like you, part one is done: nothing you have written is prose, and you have already made the three decisions the rest of the book only executes.

Part Two: The Scaffold

Three chapters. At the end of them a command turns your files into a book, and you have personally verified that the command tells you the truth.

Chapter 5: Plain Text, Git, and Why the Bible Is Separate

> Why Markdown and Pandoc rather than a word processor. The project layout. The bible is canon and never part of the build. Git as the review record. The three fiction files and what replaces them for non-fiction.

Your book is going to end up as four files: a web page you can send to a reader, an A4 PDF you can annotate, an EPUB for Kindle, and a 5.5 by 8.5 inch PDF that a printer in Kentucky turns into a paperback. All four come from the same source, in one command each, or the pipeline does not work.

That requirement rules out the word processor. Not because word processors are bad at writing, they are fine at writing, but because they are bad at everything else this project needs.

What plain text buys you

Diffs. After a review pass you will apply 40 changes across 17 chapters, and you will want to see exactly what changed, per chapter, in colour. That is `git diff`, and it is the difference between revising and hoping.

Grep. Chapter 9's sweeps are grep. Chapter 12's verification is grep. When a reviewer claims a phrase appears in three chapters, you check with a command and get a number, not an impression. A manuscript you cannot grep cannot be reviewed mechanically, and mechanical review is most of the value in this pipeline.

One source, four outputs. Pandoc reads the Markdown and the metadata and emits HTML, EPUB, and two PDF geometries. Typography lives in metadata files, not in the manuscript, so changing the trim size is one line rather than a reflow of 300 pages.

The layout

```
my-book/
├── metadata.yaml          # title, author, fonts
├── metadata-print.yaml    # paperback geometry
├── metadata-a4.yaml       # editorial proof geometry
├── Makefile             # html / pdf-a4 / epub / print
├── chapters/            # the manuscript, in order
```

```
├── bible/              # canon, NOT part of the build
│   └── review/         # every review and response
├── agents/             # the review crew definitions
├── scripts/            # install, covers, preflight
├── templates/style.css # html and epub look
└── output/             # generated, gitignored
```

You do not have to build that by hand, and this is the point to stop reading and create it. The scaffold, the review-crew definitions, the cover scripts and the installer are one repository, `ai-book-tutorial`, whose location is on the copyright page of this book; clone it once and copy the template for each book you write:

```
git clone <repo named on the copyright page> book-kit
cp -r book-kit/template ~/books/my-book
cd ~/books/my-book
```

If you would rather not clone anything, every file in the tree is small enough to create by hand from this book, and the three that carry real content are the Makefile, `metadata.yaml` and `metadata-print.yaml`.

Everything in the tree above now exists on disk, including the review-crew definitions and the cover scripts. The decision the tree leaves you is which bible file to fill first, and part one already answered it: premise, then reader, then voice.

Chapters are ordered by filename, with a two-digit prefix: `02-ch01.md`, `03-ch02.md`. Inserting a chapter between two others is a rename, which git records, and Pandoc simply concatenates in sorted order. Do not fight this with clever naming. Boring numbers, one file per chapter, no exceptions.

The bible is canon and it is not in the book

`bible/` is excluded from every build. It holds the compressed truth the manuscript must stay faithful to, and it exists because continuity does not survive 80,000 words on memory. It survives on a file you reread before every drafting session.

The rule that makes it work is short: **when a chapter contradicts the bible, you fix the chapter.** Not the bible.

That sounds obvious and it is violated constantly, because fixing the bible is one line and fixing the chapter is an afternoon. The one time changing the bible is correct is when the draft found something better, and then it is a deliberate act with its own commit and its own consequence: every earlier chapter that touched the changed fact goes back on the list. Chapter 10 covers how to decide. The point here is that both paths are visible in git afterwards, and the accident path is not available.

The fiction bible, and its non-fiction replacement

Four files are the same whichever kind of book you are writing: premise, voice, master outline, and the one-line glance. Three are fiction-only, and the swap for them is the most useful paragraph in this chapter.

A novel's bible also holds `characters.md` (continuity anchors: age, history, voice markers), `world-tech.md` (the physics of the world, and specifically what the systems **cannot** do, which is the interesting half) and `timeline.md` (the day-by-day clock, because a thriller's tension lives there). None of the three means anything for non-fiction. This book replaced them with:

- `reader-and-promise.md` instead of the cast list, because in non-fiction the reader is the protagonist. Chapter 3 built it.
- `verified-facts.md` instead of the world's physics, holding every fact the book asserts, split into two classes. **Class A** is verified on this machine, with the command and the date, and may be stated flatly. **Class B** belongs to somebody else, changes without notice, and may only appear next to the authority you check it against. Every royalty rate and spine constant in this book is Class B.
- `research-notes.md` instead of the timeline, holding each external fact, its source, and the chapter that uses it, so a reviewer can find it.

That Class A and Class B split is what stops a technical book from confidently printing a number that was true in March. It is also the rule this book broke: a blind review pass found nine Class A numbers in the prose with no row and no date in the file that is supposed to hold every one of them, including a warning count the book uses twice to justify a permanent change to its own build.

Git is the review record

Commit granularity that has worked across four books:

- one commit per bible file as it is written
- one commit per drafted chapter, message `draft: ch07 <title>`
- one commit per review pass, containing the report files only
- one commit per applied revision batch, referencing the response file

The point is not tidiness. The point is that in month five, when a reviewer raises a finding you declined in month three, `git log` plus the response files tell you that you declined it and why. Without that record you will re-litigate the same twelve arguments every pass, and you will lose about half of them by attrition.

Initialise now and commit the bible first:

```
git init
git add bible metadata*.yaml Makefile
git commit -m "bible: premise, reader, voice, outline"
```

The first commit of the project contains no prose. That is the correct shape.

The decision this chapter closes

Plain text or a word processor, and where canon lives. The project exists, git is initialised, and the bible is committed before a sentence of the manuscript.

Chapter 6: The Toolchain, and Verifying Your Own Tools

> Install pandoc, xelatex, fonts. The five make targets and when each runs. Then the real lesson: test the tools that count things. The wordcount that reported 16 words where 7 existed, the diagnosis, the fix, and make epub failing until a cover exists.

Four programs and one font family. On macOS the template installs all of it:

```
./scripts/install-macos.sh
```

It walks the chain in order, checking each step before it moves: Xcode Command Line Tools, Homebrew, MacTeX without the GUI, pandoc, git, ImageMagick, poppler. It ends by asking TeX Live whether the fonts the book actually references are there:

```
kpsewhich texgyrepagella-regular.otf
```

If that comes back empty, the script tells you the exact `tlmgr` command to fix it rather than failing at build time with a LaTeX error that names nothing. On Linux the same set comes from your package manager: `pandoc`, `texlive-xetex`, `texlive-fonts-extra`, `git`, `imagemagick`, `poppler-utils`.

The fonts deserve one paragraph, because it is a trap. This book's metadata references TeX Gyre Pagella and Heros by filename rather than by family name, plus DejaVu Sans Mono for code, for a reason chapter 15 pays for in full. They ship with TeX Live, so xelatex resolves them through kpathsea even on a machine where the operating system has never heard of them, and the GUST license permits embedding and commercial use. Use a font you found online and two things can happen: KDP rejects the interior because the font is not embedded, or the license does not allow you to sell a book containing it. Neither surfaces until upload day.

The five commands

```
make html        # browser proof, self-contained
make pdf-a4      # editorial proof, good to annotate
make wordcount   # prose words, against your target
make epub        # Kindle ebook (needs a cover)
make print       # KDP paperback interior, at trim
```

Which one you run depends on the session. Drafting: `make html` after each chapter, `make wordcount` when the session ends. Reviewing: `make pdf-a4`, then read it on a tablet away from the terminal, because you catch different things when you cannot fix them. Publishing: `print`, `epub`, `covers`, `preflight`, in that order, and chapter 15 explains why the order is not negotiable.

Run `make html` now, before you have written anything. An empty book that builds is a scaffold. A full book that has never built is a problem you get to discover at the worst possible time.

Now test the tools that count things

Here is the chapter's actual lesson, and it is not about pandoc.

`make wordcount` exists so you can measure the draft against the outline's targets. Its job is to count prose words, and prose means neither the headings, nor the drafting comments, nor the outline beat quoted at the top of each chapter file. The recipe in the template I had been using for three books did this:

```
cat $(CHAPTERS) | grep -vE '^(<!--|>|#|\\)' | wc -w
```

Read it as English: drop every line that starts with a comment marker, a blockquote marker, a heading marker, or a backslash. Which is not the same sentence as "drop comments". It drops the line a comment *starts* on.

I tested it, because writing this chapter required me to state what it does. Seven words of prose, one three-line comment, one beat:

```
# Chapter X

<!-- comment line one
hidden words on continuation line
end of comment -->

> beat blockquote skipped words

Seven real prose words appear right here.
```

```
$ make wordcount
16
```

Sixteen. Nine words of comment body counted as manuscript. On a real chapter with a multi-line note about what still needs research, the count drifts up by 20 or 30 words per chapter, which across 17 chapters is roughly 400 phantom words. Not a disaster. But it is a measuring instrument reading high, and every decision in part three is made against that instrument.

Two honest fixes exist. Keep every drafting comment on one line, which is a discipline you will violate by chapter four. Or fix the recipe, which is four lines of awk:

```
wordcount:
    @cat $(CHAPTERS) \
    | awk '/<!--/{c=1} c{if(/-->/){c=0}; next} 1' \
    | grep -vE '^(>|#|\\)' | wc -w
```

The awk holds one bit of state: inside a comment or not. Same test file, after the fix:

```
$ make wordcount
7
```

This project does both, and the template in the repository carries the fixed recipe now.

Why this is a chapter and not a footnote

Because you are about to spend three months making decisions against numbers that your tools report, and some of those tools are 90% right in a way that never announces itself. The word count was one. Here is another, from the same afternoon:

```
$ make epub
make: *** No rule to make target `assets/cover.jpg',
    needed by `epub'.  Stop.
```

The EPUB target lists the cover as a prerequisite, and the cover is gitignored, so `make epub` fails on a fresh clone until you put an image at `assets/cover.jpg`. That is arguably correct behaviour: shipping an ebook with no cover is worse. It is also a five-minute detour on the day you are already nervous, and it belongs in the book because I hit it.

The rule, and it applies to every tool you did not write:

> Before you trust a number, feed the tool an input whose answer you already know.

Seven words. One comment. Count it. That is a two-minute test and it is the difference between a measurement and a guess.

One refinement, learned late enough on this book to be worth stating plainly. A known-answer test proves the tool does what you told it to do. It does not prove you told it the right thing. When I later wrote a script to count single-sentence paragraphs, it passed its self-test and reported that a quarter of the book's paragraphs were one sentence long, which was alarming and wrong: it was counting the one-line stems that introduce a code block as paragraphs. The self-test could not catch that, because the fixture I wrote for it made the same assumption the script did. When a measurement surprises you, read the twenty things it counted before you act on the number. Do the same for the crutch-word sweep in chapter 9: put four instances of "almost" in a file and confirm the script says four. Capitalise one of them, because the first version of that sweep was case-sensitive and my all-lowercase fixture made exactly the same assumption the script did. Do it for the em-dash grep. Do it for anything a review agent will later use as evidence, because chapter 12 is about the fact that agents are confidently wrong a small percentage of the time, and an agent using a broken counter is wrong with a citation attached.

The decision this chapter closes

`make html` produces a book. `make pdf-a4` produces a proof. And you have personally run one test whose answer you knew in advance, against the tool you will believe for the next three months.

Commit the toolchain state and move on to the outline.

Chapter 7: The Outline Is a Contract

> A bad scene costs one line in the outline and the tail of the book in chapter 22. The anatomy: locked decisions, spine, per-chapter beat and end beat. Word count as part of the skeleton. The at-a-glance companion and the edit order.

A bad scene in the outline costs one line to fix. The same bad scene in chapter 22 costs you the rest of the book, because everything downstream was built on it. That ratio is the entire argument for outlining, and it holds for non-fiction at least as strongly: a chapter that does the wrong job is a chapter whose three neighbours now overlap.

The outline is not a summary of the book. It is a contract with it. Five parts.

Locked decisions

At the top, before any chapter, the non-negotiables the whole book obeys. From a novel of mine:

> The climax is a failed root ceremony and a frozen settlement, not a machine shutdown.

From this book:

> The book is the pipeline, in pipeline order. No chapter arrives out of order, even when that would be tidier.

Locked decisions exist to be cited. In month four you will want to reorder two chapters for a reason that feels excellent at 11pm. If the reorder violates a locked decision, the answer is no, and the file gives you a no that does not depend on how tired you are. Eight of them is plenty. Thirty means you have written the book in note form and will resent it.

The one-sentence spine

The premise sentence, aimed at structure instead of at a reader. It answers what changes across the whole book, so any chapter can be tested against it.

Per chapter: five fields

For each chapter, in the master outline:

- **POV and place** for fiction, **the angle** for non-fiction

- **Target word count**
- **The beat**, which is what happens, in one to five sentences
- **The end beat**, which is what has changed by the last paragraph

A beat is "a car ignites the grass on the vineyard margin and costs them nothing yet". A beat is not "they drive through the vineyard talking". The test is whether someone else could draft the chapter from it and get the same events.

No prose in the outline. If you write a good line in the outline you will spend the drafting session trying to justify it.

The end beat is the rule that saves the book

Every chapter ends with **changed danger**. No chapter ends with everyone going to sleep. If nothing has changed by the last paragraph, it is not a chapter, it is a continuation that got a heading.

The non-fiction twin of that rule, which this book follows: **by the end of the chapter, a decision is closed and a file exists that did not exist before.** A chapter that leaves the reader merely better informed has not earned the next page. That rule killed two chapters of this book at the outline stage, at a cost of two lines each. One was a chapter about choosing a model, which closed no decision the reader could act on. The other was a tour of Markdown syntax.

Write the end beats before you write the beats if you can. It is harder and it front-loads the structural work into the cheapest possible file.

Word count is part of the skeleton

A 1,500-word action beat and a 3,500-word procedural chapter are different animals, and the outline has to know which one it is asking for. Put the number in the field. Two things follow.

You get an honest total before you write, which is how you discover at zero cost that your 24-chapter outline is a 60,000-word book and your genre expects 90,000. And you get per-chapter drift as a signal during drafting: a chapter that comes in at double its target is usually two chapters, and a chapter at half is usually missing its complication.

The at-a-glance companion

The master outline is 2,400 words for this book and too long to hold in your head. So there is a second file, `bible/outline-at-a-glance.md`, with one line per chapter:

> 6. **The Toolchain, and Verifying Your Own Tools** (1,000): install, five make targets, and the word count that counted 16 words where 7 existed.

That is the file that sits beside the keyboard while drafting, and it is the file you hand a reviewer when you want an opinion on structure rather than on prose.

The two files drift, so the edit order is a rule: **change** `master-outline.md` **first, then update the glance.** Never the other way. When they disagree, the master is right by definition, and the glance gets regenerated from it. The review consequence is immediate: a reviewer reading the glance and

a drafter reading the master produce findings that are really just a stale one-line summary, and you spend an hour proving it. The line quoted above is a live example. Three independent reviewers flagged its word count, because for two rounds it was wrong.

How much to outline before drafting

All of it, at beat level, for the first book. Every chapter, POV, target, beat, end beat, before a word of prose. Once you have done that once you will know where you can safely leave a part sketched, and it is later than you think.

The reason has nothing to do with planning temperament. The review orbit in part four compares the manuscript against the outline, line by line. With no outline, the pacing pass has nothing to measure and produces taste. With an outline, it produces a table.

The decision this chapter closes

`bible/master-outline.md` and `bible/outline-at-a-glance.md` both exist. Locked decisions at the top, a spine sentence, every chapter with a target and an end beat, and no end beat that says nothing changed.

Commit them. Part two is done, the scaffold builds, and the next chapter finally writes prose.

Part Three: Drafting

Three chapters, and the first prose in the project. The rules here are about protecting the draft from you and from the model, in that order.

Chapter 8: Drafting, One Chapter Per Session

> Chapter file anatomy and why the build skips the first two parts. The full drafting prompt with every constraint's reason. One chapter per session. Never invent canon: stop and update the bible instead. What to do with the model's plot suggestions.

Now you draft, and the first thing to notice is how little of this chapter is about prompting.

The chapter file has three parts

```
# Chapter 1: Basement Duty {#ch-basement-duty}

<!-- Act 1, day 1, target 2,800 words -->

> The outline beat, quoted as a blockquote.

Prose starts here, below the beat.
```

Heading, then the drafting note as an HTML comment, then the outline beat as a blockquote, then prose. The build strips the comment and the blockquote, and so does `make wordcount`, assuming you fixed the recipe in chapter 6.

That arrangement does one useful thing: the contract travels with the chapter. When you open the file in month three, the beat you promised to write is at the top of it, and drift becomes visible instead of arguable. When a review agent reads the chapter it can compare prose against beat without opening the outline. And when you rewrite the chapter, the beat is right there to be renegotiated deliberately.

The drafting prompt, in full

This is the fiction version, from my third novel. Every line is load-bearing.

> Write chapter N of the outline. Follow `bible/voice.md` exactly. Follow the chapter beat exactly. Do not invent plot, characters, or technology that the bible does not already carry: check it against `world-tech.md`, `characters.md` and `timeline.md`. Write in close

> third person, past tense. The chapter ends with changed danger. Do not write notes to the author. Do not moralise. Do not summarise the chapter's meaning at the end. After the prose, quote my "if you remove a paragraph, the book still works" test and give me the three paragraphs you would cut first.

Why each constraint is there:

"Follow voice.md exactly." Without the pointer the model uses its house style, which is the average of everything. This is the sentence that makes part one pay.

"Do not invent plot, characters, or technology." A model that needs a detail will invent one, plausibly, and it will contradict chapter 4 in a way you notice in chapter 19. Naming the canon files converts invention into lookup.

"Close third person, past tense." Point of view drifts across a long chapter otherwise, usually into a paragraph of omniscient summary right where the tension should be.

"Ends with changed danger." The end beat, restated at the moment of writing.

"Do not write notes to the author." Otherwise you get "[Note: consider expanding this scene]" inside your manuscript, and one of those survives to the proof copy. Ask me how I know.

"Do not moralise. Do not summarise the chapter's meaning." The two most recognisable AI habits in fiction. The model wants to land a closing sentence explaining what the chapter was about. That sentence is always the first thing a human editor cuts.

The cut-three-paragraphs request. This is the only place in the drafting session where the model is asked to judge, and it is asked to judge subtractively, which is the one aesthetic question models are reliably good at. You will take about one suggestion in three, and the exercise tells you where the chapter is padded before you have grown attached to it.

For non-fiction, the same prompt with three substitutions: the canon files become `reader-and-promise.md` and `verified-facts.md`, close third person becomes second person for procedure and first person past for evidence, and changed danger becomes "the chapter closes a decision and names the file the reader now has". The four negative constraints do not change at all.

One chapter per session

Not because of context windows. Because of accountability.

A session that drafts one chapter ends with one file, one `make html`, one wordcount check, and one commit. When the review orbit flags that chapter in month three, you can read the diff and remember the decision. A session that drafts six chapters ends with a blur, and it will contain two chapters you never actually read.

The session shape that has worked, four books running:

1. Reread the beat and the previous chapter's last two paragraphs.
2. Draft.
3. Read it, on screen, once, without editing.
4. `make wordcount`, compare against the target.
5. Commit as `draft: chN <title>`.
6. Stop. Do not start the next chapter because you feel warm.

Point 3 is the one people skip and it is the reason to keep sessions small: the only way an unread chapter reaches a reviewer is if you never read it.

The draft never invents canon

The most important rule in part three. When the draft needs a fact the bible does not have, you stop drafting.

You do not let the model decide the surname, the compression algorithm, the distance between two towns, or what happens to be true about the reader's toolchain. You add it to the bible, commit that, and then continue. It costs two minutes and it is the difference between a book with continuity and a book with seventeen small contradictions you find in the mechanical review at the worst possible moment.

This applies with even more force to non-fiction, where the equivalent of a continuity error is a false claim. If the chapter needs a number, it goes through `verified-facts.md` first, with the command that produced it and the date. If the number belongs to Amazon, it goes in as Class B, next to the page you check. No number reaches prose without one of those two paths. That rule is why this book prints "16 words where 7 existed" and refuses to print a royalty rate.

What to do with the model's plot suggestions

They will arrive unrequested, and some of them are good. The test is not whether the idea is good. The test is:

> Does the suggestion survive being written into the bible?

Open the canon file, write the change as a canon fact, and look at what it breaks. If it breaks nothing and improves the beat, take it, commit the bible change first, then draft. If it breaks three earlier chapters, the honest cost is those three chapters plus the review pass, and now you are choosing with the price tag visible.

What you never do is take the suggestion inside the prose and leave the bible alone. That is how a book acquires two contradictory truths, both written confidently, forty pages apart.

The decision this chapter closes

Chapter one exists as prose under its beat. `make html` shows it in a browser, `make wordcount` compares it against the outline, and git has it as one commit with your name on it.

Sixteen more times, and then part four takes it apart.

Chapter 9: Keeping the Prose Human

> The two sweeps, run every few chapters. Crutch words with a rate and a verdict. AI tells as a density problem, the watch list, and the structural fix. The em-dash grep. The two health tests.

Two sweeps, run every three or four chapters, never saved for the end. Both take under a minute. Both produce a number and, more importantly, a verdict.

Sweep one: crutch words

Crutch words are the softeners that drain force out of a sentence without announcing themselves. Ten of them cover most of the damage:

```
for w in almost very quite rather somehow slightly \
         really perhaps seemed somewhat; do
  echo "$w: $(grep -riwo "$w" chapters/ | wc -l)"
done
```

Note the `-i`. The first version of that loop did not have it, and so it could not see `Almost` at the start of a sentence. One live hedge hid behind that for three revision rounds, in this chapter, which is the chapter about sweeps.

Raw counts are not the output. Two things turn counts into a decision.

A rate. Counts per 1,000 words, because 40 instances of "almost" means something different in 30,000 words than in 90,000. Divide by the wordcount from chapter 6, the fixed one.

A verdict. Here is my own sweep on an 83,000-word manuscript, in full:

> `almost` 0.52 per 1000w, `very` 0.59 per 1000w. Both slightly high for a thriller. Not worth your time now; handle at copy-edit.

That verdict is the useful artefact. It says the number is imperfect and the fix is not due yet, which stops you spending a week on softeners while the third act has no antagonist. A sweep that reports "clean" saves you the same week. A sweep that reports numbers with no verdict guarantees you will fiddle, because numbers on a screen ask to be lowered.

This book's own sweep is a worked example of why the verdict matters more than the count. It reported `almost` at 0.74 per 1000 words, high enough to look like a problem. Most of the hits were the word being discussed rather than used, in this chapter and in chapters 1 and 6, because a book about crutch words is full of crutch words in quotation marks. Today the sweep finds 13 and 10 of

them are quoted, which leaves three live hedges at 0.17 per 1000 words. Two earlier ones were the same vague phrase describing the same review finding in two chapters, and both were replaced with what the finding actually said; one hid from the sweep entirely until a blind pass found the sentence-initial `Almost` the case-sensitive loop could not see.

A sweep counts strings. You supply the reading, and you check the sweep's own blind spots before you quote its number. Note also that these counts move every time the chapter about them is edited, which is why the sentence above gives the reading and not just the figure.

Test the sweep before you trust it, as chapter 6 insisted: put four instances of "almost" in a scratch file and confirm the loop says four. Note the `-w` in that grep. Without it, "almost" matches inside other words and "very" matches "every", and your counts are junk in the direction that makes you feel bad.

Sweep two: AI tells, which are a density problem

This is the sweep that decides whether the book reads generated. The critical insight, and it took me a full book to internalise:

A single instance is fine. Density is the defect.

"Not X. Just Y." once in a novel is a rhetorical move a human would make. Once a page is a machine signature. Same for every item on the watch list from `bible/voice.md`:

- `Not X. {Not / Just / But} Y.` contrast structures, the most recognisable tell
- `a kind of` and `some kind of` hedges, which can usually just be deleted
- `it was as if` and `as though` similes
- `something in the [way / voice / set of his shoulders]`
- `in the silence that followed`, as a transition crutch
- `for a moment` and `for a long moment`, temporal hedging
- `there was a [tension / stillness]`, where the noun should be doing the verb
- `the weight of [what she had done]`
- `a quiet [noun]`, as in "a quiet certainty"
- `still` as an intensifier and `just` as a softener, used as tics
- the staccato three-beat `Not X. Not Y. Just Z.`

Grep for each, note where the hits cluster, and go to the clusters. Three of these inside two paragraphs is the finding. One on page 40 and one on page 200 is prose.

The fix is structural, never a word swap

When you find a cluster, the instinct is to swap words: delete "a kind of", replace "for a moment" with "briefly". That produces prose with the same skeleton and slightly different skin, and it will still read generated, because the tell was never the vocabulary. The tell is the rhythm: stacked short fragments, each carrying one clause, in a row.

The fix is to fold them into one longer sentence that carries a subordinate clause.

Before:

> She looked at the panel. Not surprise. Something closer to recognition. For a moment she did not move.

After:

> She looked at the panel with the flat attention of someone recognising a thing she had been told to expect, and she did not move.

The second version is longer, more specific, and could not have been assembled from fragments. Your long, specific sentences are the human signature in current AI prose, and they are the defence. Lean on them.

The em-dash grep

The highest-return single command in the pipeline:

```
grep -c '—' chapters/*.md
```

Zero on every file, or you have work to do. The em-dash is the loudest machine tell in circulation, and readers who could not name a single other pattern are calibrated on that character.

This book ran that grep as part of its own review. Zero hits in narrative prose, which is what the rule governs, and three hits in the manuscript as a whole: every one of them inside a fenced code block, where the character is the argument to the grep that hunts it. The response file records it in those words rather than reporting a clean zero.

An earlier draft of this paragraph claimed two. It was true when written and went stale when a later chapter printed the same command, and the voice review caught it as the only blocking finding in the orbit. A book about verification that prints an unverified number about itself is the exact failure this chapter is about, so the correction is in the response file rather than quietly applied.

If you use an AI to draft, expect em-dashes in the first output of every chapter and fix them as part of the drafting session, not at the end. Splitting a sentence in two, or reaching for a colon, changes the rhythm, and rhythm changes are cheap in the drafting session and expensive after the pacing review has read the chapter.

The two health tests

From the voice guide, run on any scene that feels wrong but reads fine:

> **Remove the technology nouns.** The scene must still contain a person trying to do something, another pressure resisting them, a discovery, and a changed choice.

If deleting the machines empties the scene, the scene was a demo. For non-fiction, remove the tool names: the paragraph must still hold a decision, a reason, and a consequence, or it was a tool tour.

Remove the final interpretive sentence. If the consequence remains legible, leave the sentence out.

Nine times in ten it stays out. The model writes that sentence because it is trained to land a point, and the chapter had already landed it.

When to run the sweeps

Every three or four chapters, and once more before the review orbit. Not at the end. The reason is not thoroughness, it is calibration: you want to catch your own recurring tic in chapter 5 so that chapters 6 through 17 do not have it. I found a `for a moment` habit on chapter 6 of one book and it cost me an afternoon. Finding it at chapter 30 would have cost a week.

The decision this chapter closes

Two sweep reports exist, each with a rate and a verdict, saved in `bible/review/`. At least one cluster has been fixed structurally rather than by word swap, and the em-dash grep returns zero.

Then back to drafting, until the draft fights back.

Chapter 10: When the Draft Fights Back

> The failure modes of a real draft and the response to each. The stuck chapter, the sprawling chapter, the draft that wants to leave the outline. Editing while drafting and what it costs by chapter 30. Word count against target.

Somewhere between chapter 6 and chapter 11 the draft stops cooperating. This happens every time, in every book, and none of it is a sign that the idea was wrong. Each failure has a shape, and each shape has a cheaper response than willpower.

The chapter that will not start

You reread the beat, you draft, and the output is limp. Three attempts and you have three limp openings.

Almost always the beat is two beats. It contains a discovery and a confrontation, or a procedure and its consequence, and the chapter cannot decide which one it is about. Splitting it into two chapters usually fixes the paralysis inside ten minutes, and the outline absorbs the change with one renumbering.

The second most common cause: the chapter has no pressure in it. The character wants nothing this hour, or in non-fiction, the reader has no decision to make here. Chapter 7's rule applies retroactively. If you cannot name the decision the chapter closes, the chapter should not exist, and the honest fix is a merge with its neighbour rather than a better prompt.

The chapter that sprawls

You aimed for 2,800 words and produced 5,200, and it is not bad, it is just not ending.

A sprawling chapter is a chapter with no end beat. The prose keeps going because nothing has told it that the job is done. Reread the end beat field in the outline. If it is empty or vague, that is the bug, and you write the end beat before you cut a word.

Do the cutting from the middle, not the end. The end is usually where the chapter finally arrives; the middle is where it circled. This is exactly what the "give me the three paragraphs you would cut first" request in chapter 8 is for, and a sprawling chapter is the one place I take that suggestion nearly every time.

The draft that wants to leave the outline

The interesting failure. Halfway through your chapter 12 the draft finds something better than what you planned: a relationship you did not design, an argument that belongs three chapters earlier, a machine that is more frightening than the one in the bible.

Do not follow it silently, and do not refuse it on principle. Run the test from chapter 8:

> Does the change survive being written into the bible?

Open the canon file and write the change as a fact. Then look at what it breaks. Three outcomes, all of them fine:

It breaks nothing. Take it. Commit the bible change, then continue drafting. This is the good case and it happens more than you expect, because the outline was written before you knew the book.

It breaks two or three earlier chapters. Now you have a price. Write it down: which chapters, roughly how much work, and what the change buys. Then decide, and put the decision in the bible as a locked decision with a date. Half the time the answer is yes and half the time it is "not in this draft", and either is fine as long as it is written.

It breaks a locked decision. The answer is no, unless you are willing to reopen the decision explicitly and accept that the whole book downstream is now in question. I have done this once, in book two, and it cost five chapters. It was correct, and what made it survivable was that the reason is in the bible, so the following four reviews stopped arguing about the ending.

The path that ruins books is the fourth one, which is following the better idea in the prose while the bible still says the old thing. Then you have two truths, both confident, forty pages apart, and neither the model nor you will remember which was intended.

Editing while drafting is the deadliest anti-pattern

Write a chapter, then improve it, then write the next one, then improve that. Each improvement is defensible. Each one also pulls the book a degree off the spine, because you are optimising a chapter against your taste this morning rather than against the outline. By chapter 30 you are writing three books, and the review orbit will tell you so in a way that costs a month.

The rule is the one from chapter 1, and it is the whole reason the pipeline is shaped this way: **draft with one process, review with another.** During drafting you may fix a factual error, a name, or an em-dash. You may not restructure, you may not rewrite yesterday's chapter, and you may not "just tighten" your chapter 3 because you now know how your chapter 9 goes. Write it on a note in the response file instead, where part four will find it.

Same for the model. If your agent offers to revise the previous chapter, decline. A revision at drafting time is a revision with no report behind it, which means it is untraceable, and untraceable changes are what make the review orbit produce findings you cannot explain.

Word count against target, honestly

Run `make wordcount` at the end of every session and compare to the running total the outline promised. Two failure modes and one correct response each.

Running short. A 95,000-word plan that is 68,000 words on page 300 is not shorter. It is missing scenes a reader paid for. When you find yourself 25% under target, the diagnosis is almost never prose density. It is that beats are being summarised rather than dramatised, or that procedures are being asserted rather than walked through. Both are visible per chapter, and both are fixed by writing the missing scene rather than by padding the existing one.

Running long. Less dangerous. Usually two or three chapters carrying double weight, which you already know how to diagnose.

Boring momentum rules

None of these are inspiring, and all four have survived four books.

- Draft in the same slot every day, even a short one. The model does not care, but the reread does.
- Never end a session mid-chapter. End with a committed chapter or with a note about the first sentence of the next one.
- Do not read the whole manuscript while drafting. You will edit, and part three forbids it.
- When you are stuck for two sessions, go back to the outline. The problem is upstream. It has been upstream every time.

The decision this chapter closes

A complete first draft exists, every chapter is committed, and every place the draft left the outline has a written decision in the bible with a date on it.

Nothing has been reviewed yet. That is next, and it is where the book actually gets good.

Part Four: Review

Four chapters. The draft is finished and unreviewed, which means it is not yet a book. This part turns opinions into a work plan you wrote yourself.

Chapter 11: The Review Orbit

> One agent per dimension, blind. The six dimensions, fiction and non-fiction mapping side by side. The shared report anatomy. Why blindness is the design. How to run it with subagents and how to run it without. The tension ledger.

The review orbit is the part of the pipeline that has no affordable non-AI equivalent, and it is the reason my third book took six weeks instead of four months.

The idea is simple enough to state in one sentence: **one reviewer per dimension, each blind to the others, each producing its own report file.**

The six dimensions

Six passes, and each one is forbidden from doing the other five jobs.

Report	Fiction question	Non-fiction question
`tension.md` / `momentum.md`	Does the pressure rise?	Does each chapter earn the next page?
`pacing.md`	Rhythm, chapter lengths, scene vs summary	Same, plus procedure vs explanation
`voice.md`	Violations of the voice guide, AI tells, em-dashes	Identical
`logic.md` / `accuracy.md`	Causality, plants and payoffs, competence	Does every command run? Is every claim sourced?
`mechanical.md`	Everything countable: dates, numbers, names	Identical
`copyedit.md`	Line-level prose, cut candidates	Identical

Three of those transfer unchanged, which is why the same crew works for both kinds of book. Two need a real translation, and the translation is the interesting part.

Tension becomes momentum. A thriller's tension pass builds a curve of threat. A non-fiction momentum pass builds a ledger: for each chapter, what decision does it close, and what does the reader leave holding? An empty cell in the decision column is the same defect as a flat stretch in a tension curve, and it is just as mechanical to spot.

Logic becomes accuracy. A novel's logic pass asks whether chapter 9 reacts to evidence chapter 6 established. A non-fiction accuracy pass asks whether the command on line 41 actually runs, and

whether every number has a source with a date. It is the harshest pass in the non-fiction orbit, and the one that found most of what was wrong with this book.

The definitions live in `agents/`, one file per angle, in the template. They are plain Markdown, so they drop into Claude Code as subagents, into opencode as agents, or into any tool that will read a file and follow it.

Why blind

Each reviewer gets the manuscript and the bible. Never the other reports.

Blindness buys one thing, and it is worth the extra runs: **independent agreement becomes evidence.** When three reviewers who have never seen each other's work all flag chapter 14, that is not taste. You cannot dismiss it as one reviewer's hobby horse. When one flags it and the others do not, that is a data point about the reviewer as much as about the chapter.

Show reviewer two what reviewer one said and you get agreement, immediately and politely, because that is what these models do. You have then paid for six reviews and received one, laundered through five confirmations. This failure is silent. It looks like consensus.

Running the orbit with subagents

In Claude Code, one prompt does it:

> Run the full review orbit on the manuscript. Spawn separate subagents, one per dimension: momentum, pacing, voice, accuracy, mechanical, copyedit. Each agent reads `chapters/` and `bible/` and never any file in `bible/review/`. Each writes its own report to `bible/review/<dimension>.md` using the shared anatomy: verdict, findings grouped Blocking / Should fix / Minor, with file:line citations. Do not reconcile yet.

Six agents, running in parallel, on a full manuscript. The last instruction matters: reconciliation is a separate act, done in chapter 12, by you.

Running the orbit without subagents, and what that costs

You may not have subagents. The workaround is to run the passes sequentially, one fresh context per dimension, and enforce blindness by hand:

1. Start a new session or clear context.
2. Load only `chapters/`, `bible/`, and one agent definition.
3. Write the report to `bible/review/<dimension>.md`.
4. Discard the context before the next pass.

What you must not do is run six dimensions in one continuous conversation. By dimension four the context holds the first three reports and you are back to laundered consensus.

I can tell you what the sequential version costs, because this book ran both. The first orbit was six passes inside one context: procedurally blind, since no pass quoted another, but not provably

blind. It raised 24 findings, two of them blocking, and six of its 23 countable claims did not survive being counted.

Then the same manuscript, after two rounds of fixes, went through six genuinely isolated agents. That orbit raised 31 findings, **nine** of them blocking, and only two of its 29 countable claims failed verification. Three of the nine blocking defects were things I had introduced myself while fixing the first orbit, which no amount of self-review had caught. It also produced something the sequential version cannot produce at all: three items flagged independently by passes that had never seen each other's work.

Sequential passes are worth running. They are not the same instrument, and the difference is roughly a factor of three in the wrong-finding rate. Note the deviation in your response file, then close it when you can.

The report anatomy

Every report, every dimension, same shape:

```
# {Dimension} review: {Book title}
## Verdict          (2-3 sentences, rating first)
## {Dimension-specific tables}
## Findings
   Blocking / Should fix / Minor, each one:
   file:line, then the finding
## One-line summary for the author
```

The uniformity is not neatness. It is what makes reconciliation possible in chapter 12: you are merging six documents with the same skeleton, so overlaps line up by severity and by citation instead of by prose.

One rule belongs with the anatomy rather than with the argument: **no reviewer may rewrite prose.** Findings only, with citations. A reviewer that edits is a reviewer whose findings you can no longer count, because the evidence has been altered by the witness, and the shared report format is what makes findings-only practical.

The **one-line summary** at the bottom is worth its own instruction. Forcing each reviewer to name the single most useful sentence it produced is how you find the finding that matters among forty that are technically correct.

The tension ledger, the most actionable table in the pipeline

For fiction, one table beats the rest of the orbit combined: one row per chapter, with threat present, who is under pressure, a score out of ten, and a note. It turns a feeling into a shape you can see.

Here is what a blind tension pass wrote about my second novel:

> The book has two-and-a-half acts of rising pressure and a fourth act of falling action mislabelled as a climax sequence.

That single sentence changed the ending of the book, and what made it actionable was the table under it: scores of 8, 9, 9 through the middle, then 6, 5, 6 across the last four chapters. That is a fact about the manuscript, not a preference.

The non-fiction equivalent is the chapter ledger from the momentum pass. On this book it flagged chapters 13 and 14 as a pair reading like one chapter split in two, which is not something either chapter can show you from the inside. Note what it did not catch: the two chapters this book killed for closing no decision died at the outline stage, months earlier, killed by the end-beat rule for the price of two lines. The ledger is a net for what survives the outline, not a substitute for the outline's own rule.

When to run it

Twice, at minimum.

Early, on the outline and the first three chapters. The findings are cheap to apply and they catch structural problems while a structural problem is still one line in a file. This is the run I did not do on book one.

Formally, on the complete draft, as an event: all six dimensions, one reconciliation, one response file, one commit.

The decision this chapter closes

Six report files exist in `bible/review/`, none of which has read another, all with the same anatomy, each with a verdict and a one-line summary.

Do not apply anything yet. Half of what is in those files is wrong or already declined, and the next chapter is how you tell which half.

Chapter 12: Reconcile, and Verify the Countable

> Reviews are input, not verdicts. Merge and mark NEW / STILL UNFIXED / PREVIOUSLY REJECTED. Then verify every countable claim before applying it. The three findings that did not survive, and the one that would have introduced a factual error into a correct book.

Six reports, maybe eighty findings, and a strong temptation to start fixing at the top of the first file. Do not. Reconciliation comes first, and it is the step where this pipeline earns its keep.

Two jobs, in order: merge, then verify.

Merge, with three labels

Every finding across all six reports gets folded into one file, `bible/review/reconciled.md`, grouped by severity, and each item gets exactly one of three labels.

NEW. Not raised by any previous pass.

STILL UNFIXED. Raised by an earlier pass, still present, with the earlier file cited. This label is the reason the review archive exists. Good advice dies quietly in round five otherwise: a reviewer raises it, you are busy, it does not get applied, and it never gets raised again because the next pass has different random attention. The label converts silence into a visible repeat.

PREVIOUSLY REJECTED. Raised before and deliberately declined, with the prior reasoning restated. Without this label you re-litigate the same twelve arguments every pass, and you lose about half of them by attrition, which means your book slowly becomes the average of five reviewers' preferences.

Consensus and contradiction belong in the merged file too, and they are the two entries a later pass will thank you for.

Consensus. Where three or more blind reviewers flagged the same chapter, mark it. That is your strongest signal in the entire pipeline, and it should be at the top of the file regardless of what any individual severity rating said.

Contradictions. Where two reviewers want opposite changes, mark it and say so plainly. The pacing pass asking for compression and the momentum pass asking for a scene in the same chapter is not noise. It usually means the chapter is doing two jobs, which is the chapter 10 diagnosis, and neither reviewer could see it from their single dimension.

Then verify every countable claim

This is the part nobody writes about, and it is the part that protects the book.

A review finding that can be counted must be counted before it is applied. Not because the models are careless, but because they are confidently wrong a small and consistent percentage of the time, and a wrong finding arrives with a citation and a plausible severity rating attached.

What countable means in practice:

Four claims, four commands, four answers, run against this manuscript as I write:

```
grep -rn "the weight of" chapters/ | wc -l
# 3, all three inside quoted watch lists.  INVALID

grep -rn "2026-03-1" chapters/ | wc -l
# 1, and it is this code block.            INVALID

grep -c '—' chapters/12-ch09.md
# 1, inside a fenced code block.           INVALID

make wordcount
# a number, against the outline's target.  CHECKABLE
```

Three of those four claims died on contact with a command, and I did not have to be right about anything to kill them. I had to run four greps and read twelve lines.

The rule, and it is absolute: **a finding whose countable claim does not verify is marked Invalid in the reconciled file, with the evidence, and is not applied.** Not softened, not partially applied. You write down what you checked and what you found, and you move on.

Three findings that did not survive

On the eighth-draft orbit of my second novel, three findings failed verification. Two were harmless: a claimed repetition that existed once, and a claimed timeline contradiction that came from misreading a flashback.

The third was not harmless. A reviewer flagged a technical detail as an error and supplied a correction. The correction was wrong, and the book was right. Had I applied it on the strength of the citation and the confident tone, I would have introduced a factual error into a correct book, in a technical thriller whose entire market is readers who would catch it.

That finding is why this chapter exists, and why it comes before the revision chapter rather than after it. One in twenty-odd findings, in my experience across four books, is wrong in a way that makes the book worse. You cannot tell which one by reading it. You can tell by counting.

Verification for non-fiction is harsher

For a novel, verifying the countable means counting words, dates and names. For non-fiction, the accuracy pass produces findings about commands and claims, and verification means running

things.

On this book, the accuracy pass had to be answered with actual output. When it questioned the word-count behaviour, the answer was not an opinion. It was a scratch file with seven prose words, one three-line comment, and one beat, run through the target twice: 16 before the fix and 7 after. Both numbers are in chapter 6 because both numbers were produced rather than asserted.

The equivalent discipline for a claim: every Class A fact in `bible/verified-facts.md` carries the command that produced it and the date it was run, and any finding that disputes a Class A fact gets the command re-run before anything changes. Every Class B fact, meaning anything owned by Amazon or another third party, cannot be verified by you at all, which is why the rule is that those never appear as flat assertions. A reviewer flagging a Class B number as out-of-date is almost certainly right, and the fix is not a new number, it is the authority and a check instruction.

What reconciliation is not

It is not a vote. Six reviewers agreeing that your ending is soft is strong evidence about the ending. It is not a mandate. They are six instances of similar training with similar preferences, and the aggregate has a house style of its own. The pipeline's own bias should be visible to you: it likes compression, it likes explicit stakes, it likes a closing line that lands. Sometimes your book should not do those things.

It is also not the moment to decide what to do. Reconciliation produces a verified, labelled, deduplicated list. The deciding happens in the response file, which is the next chapter, and keeping the two acts separate is what stops "reconciling" from becoming an afternoon of unrecorded editing.

The decision this chapter closes

`bible/review/reconciled.md` exists. Every finding is labelled NEW, STILL UNFIXED or PREVIOUSLY REJECTED. Consensus items are at the top. Contradictions are named. Every countable claim has been counted, and at least one item is marked Invalid with the evidence beside it, because if none is, you did not verify, you skimmed.

Commit it before you change a word of prose.

Chapter 13: The Response File Is the Work Plan

> Three dispositions and no fourth. Why the written refusal is what makes it a decision. The verification section as part of the record. This book's own response file.

Every finding that survived verification now gets exactly one of three dispositions, written down, in `bible/review/response.md`:

Applied. Concretely, with what changed.

Deliberately not applied. With the reason.

Deferred. Parked for a later pass, which is different from declined and should never be used as a polite way of declining.

There is no fourth disposition. In particular there is no "noted", because noted is what you write when you intend to think about it later and will not.

Why the refusal is the important one

An unwritten refusal is not a decision. It is a thing you happened not to do, and it has no defence. Three passes later a fresh reviewer raises the same point, you have no record of having considered it, and you either apply it out of guilt or argue it again from scratch.

A written refusal, from my second book's response file:

> **Deliberately not applied.** The Chen depot accusation does not escape publicly. Option 2.1(b) is a larger rewrite; this pass chose 2.1(a) and 2.1(c) instead.

That entry does three things. It states the decision, names the alternative that was rejected, and says which pass made the call. When the same finding came back in the next orbit, marked PREVIOUSLY REJECTED by the reconciler, the argument took four minutes instead of an afternoon.

A review is advice. You are the editor of last resort, and the only way that authority survives contact with five confident reviewers is if you write your refusals down at the moment you make them.

The verification section

The response file does not end with dispositions. It ends with evidence:

```
## Verification
- git diff --check
- make html epub wordcount
- source scans for every blocking phrase, and for
  every locked style-sheet violation
```

Three lines, and each is a command with an output you looked at. `git diff --check` catches whitespace damage from bulk edits, which is exactly the kind of thing that survives into a print interior as a broken line. The builds prove the manuscript still compiles to all four outputs, because a revision that breaks `make epub` breaks it silently until publishing day. And the source scans prove each blocking finding is actually gone, phrase by phrase, rather than believed to be gone.

Believed-to-be-gone is the normal state of a revision without this section. You applied 40 changes in one sitting, and two of them you thought you had made.

This book's response file

The same discipline, on the book you are holding. From `bible/review/response.md`:

The applied items included the front-matter fix (a fiction disclaimer inherited from the novel template, which would have been a strange thing to print in a non-fiction book), the word-count recipe, and a corrected claim in chapter 9 about its own em-dash count.

Every entry in that list is a chapter of this book you have already read or are about to: the em-dash count from chapter 9, the word-count recipe from chapter 6, the font from chapter 15. That is what an applied section looks like when the response file is doing its job. It is a table of contents for the work rather than an argument for it.

The deliberately-not-applied section is where the interesting entries live, and on this book it held exactly one that no edit could close: the first orbit ran as sequential passes in one context rather than as six isolated agents. Written down as a deviation, it stayed visible for two more rounds until it could actually be fixed, and when it was, the isolated orbit found nine blocking defects that the sequential one had missed, three of which I had introduced myself while fixing the first orbit's findings.

That is the argument for the section. A written refusal is a to-do item with a reason attached, and it survives you forgetting about it.

The response file is the work plan

Once it is written, revision is mechanical, which is the entire point. You are no longer deciding while editing. You are executing a list you already decided, in one sitting, and the list has a verification section that tells you when you are done.

Commit the response file before you apply any of it. The order matters: decisions first, then edits, so the git history shows a plan followed rather than a plan reconstructed.

The decision this chapter closes

`bible/review/response.md` exists, every surviving finding has one of three dispositions, at least one entry is a refusal with a reason, and the verification section lists the commands you are about to run.

Now do the work.

Chapter 14: Revision and the Gate

> Revision is mechanical once the response file exists. Apply, rebuild, re-sweep, re-scan. Then the gate as a checklist that passes or does not. The commercial review, once, before the gate closes.

Chapter 13 decided what to do. This chapter does it, and then answers a different question: is the book finished?

Revision, with a response file in hand, is four commands and an afternoon.

1. Apply the Applied items, in file order rather than in severity order, so you touch each chapter once.
2. Rebuild: `make html epub print wordcount`.
3. Re-run the sweeps from chapter 9. Crutch words, AI tells, the em-dash grep.
4. Re-scan for every blocking phrase the mechanical and accuracy passes flagged, and confirm each one is gone with a command rather than with a memory.

Step 3 exists because revision introduces tells. You rewrote 40 passages with a model's help, in a hurry, and some of them came back with a "for a moment" and two new em-dashes. The sweep after revision catches more than the sweep before it, in my experience every time.

Then commit the whole batch with a message referencing the response file, so the git history reads: reports, reconciled, response, then the revision that executed it.

The gate

The gate is a checklist, and it either passes or it does not. It is deliberately short, and every line is a command or a fact rather than a judgement:

- ☐ Crutch words inside genre tolerance, **with the verdict written down**
- ☐ Zero em-dashes in narrative prose, and any code-block instances explained
- ☐ Every review finding dispositioned, not silently ignored
- ☐ `git diff --check` clean
- ☐ `make html`, `make pdf-a4`, `make epub`, `make print` all succeed
- ☐ `make wordcount` within range of the outline's target, or the gap explained
- ☐ A non-AI human has read the opening chapter cold
- ☐ The blurb matches the book the reviews describe

The human who reads chapter one cold is not optional and cannot be replaced by any number of agents. You are looking for one thing: do they keep reading past the end of the chapter without

being asked. The reason no agent can do this is that an agent has no attention to lose. It will always finish.

The blurb matching the book the reviews describe is a real gate, not a sentiment. Take the one-line summary from each of the six reports, then read your back-cover draft. If the reviews say your book is a procedural about verification and the blurb promises a fast-paced adventure, one of the two is lying, and the reader who finds out will say so publicly.

The commercial review, once

Before the gate closes, run one pass with entirely different eyes, defined in `agents/commercial-reviewer.md`. It does not review craft. It asks the questions the dashboard will ask in chapter 16:

- What shelf is this on, and which three books sit next to it?
- Who buys it, where do they browse, what did they buy last?
- What does the back cover promise, and does the table of contents deliver what that promise requires?
- Does chapter 1 make a stranger read to the end of it?

Run it late, because it is the one pass that benefits from the finished book, and run it once, because its findings are marketing decisions rather than manuscript defects. On my second book, the commercial pass sized the manuscript against its genre shelf and flagged two things a gatekeeper would bounce, both of them in the first ten pages, neither of them visible to any craft reviewer.

Publishing blind is how good books end up under honest, boring jackets.

What a red line means

A gate line that fails is not a reason to delay indefinitely. It is a reason to do one of two things, both of which are written down.

Fix it, or accept it in writing. "Word count is 14% under the outline's target because two chapters were merged; accepted" is a legitimate gate entry. "Word count is under target" with no note is an open wound that will reappear as a one-star review about the book feeling thin.

The decision this chapter closes

The gate is green, or you know exactly which line is red and why you are shipping anyway. Either way it is written down: the filled checklist goes into `bible/review/response.md` under its own heading, with a line per gate item and the verdict beside it. A gate you did not record is a gate you will have to re-argue with yourself at 2am on upload day.

The manuscript is finished. Everything after this point is files, geometry, and a dashboard, and it is where most first-time authors get their unpleasant surprises.

Part Five: Publishing on Amazon KDP

Three chapters. The manuscript is done and the surprises are not. This part is geometry, a dashboard, and two things you can only learn by uploading, both marked where they arrive.

Chapter 15: Four Files From One Source

> What KDP wants and the four targets that produce it. Trim in three places. The cover script's arithmetic and why KDP's calculator is the authority. Preflight checks only what it can verify. The tree diagram that vanished from the print PDF.

Nothing in this part is specific to non-fiction. A novel's interior, cover wrap, spine arithmetic and dashboard answers are identical to this book's, which is why the same four targets built all three of my paperbacks.

Amazon KDP wants four files, and it wants them separately.

For the ebook: an EPUB, and a cover image. For the paperback: an interior PDF at trim size, and one cover PDF containing the back, the spine and the front as a single wrap.

Four targets produce them from the same Markdown you have been writing:

```
make print      # -> output/book.pdf   interior
make epub       # -> output/book.epub  Kindle ebook
make covers     # -> ebook JPG + paperback wrap PDF
make preflight  # mechanical validation of all of it
```

The order is not negotiable, and the reason is in the next section: the cover cannot be built until the interior exists, because the spine width is a function of the page count.

Trim size lives in three places

This book is 5.5 by 8.5 inches, which is a common trade paperback size and a safe default for a technical book. That number appears in exactly three places, and all three must agree:

1. `metadata-print.yaml`, as `paperwidth` and `paperheight`
2. `scripts/build-covers.sh`, as `TRIM_W` and `TRIM_H`
3. the KDP listing, in a dropdown, on upload day

Disagreement between the first two produces a cover that does not fit an interior, which KDP rejects with a message about dimensions. Disagreement with the third produces an accepted upload and a printed book with the wrong margins. That third failure is the expensive one, because you find it holding a proof copy.

The interior, and the diagram that disappeared

`make print` builds the interior with xelatex at trim size, with mirrored margins: 0.8 inch on the inside

for the gutter, 0.6 outside, 0.75 top and bottom. The inside margin is larger because the binding eats part of it, and KDP's minimum gutter grows with page count in bands that KDP publishes and this book will not reproduce. Look up your final page count in KDP's margin table and widen `inner` if the table asks for more than the 0.8 inch set here.

Now the part I did not expect. The first `make print` of this book succeeded, and buried in its output were 71 warnings:

```
[WARNING] Missing character: There is no └ (U+2514)
    in font [texgyrecursor-regular.otf]
[WARNING] Missing character: There is no — (U+2500)
    in font [texgyrecursor-regular.otf]
```

The mono font this template shipped with, TeX Gyre Cursor, has no box-drawing glyphs. Every directory tree in chapters 5 and 6 was silently dropped from the paperback interior. The HTML and the EPUB were fine, because they use system fonts in a browser. The build did not fail. It produced a valid, uploadable, 106-page PDF with holes in it where the diagrams were.

The fix was one line in `metadata.yaml`, switching the mono font to DejaVu Sans Mono, which ships with TeX Live and covers those code points:

```
monofont: "DejaVuSansMono.ttf"
```

Then the warning count is zero, which is the number you check:

```
make print 2>&1 | grep -c 'Missing character'
# 0
```

Two lessons, one specific and one general. The specific one: if your book contains box drawings, arrows, or anything outside Latin-1 in a code block, verify the font has it before you trust a successful build. The general one is chapter 6's rule again, arriving late and in a new costume. A warning is not a failure, and a successful build is not a correct book.

The covers, and the spine arithmetic

`make covers` does two jobs. It resizes the front art to 1600 by 2560 pixels for the ebook, which is the 1 to 1.6 ratio Amazon prefers, and it builds the paperback wrap.

The wrap is where the arithmetic lives. **Bleed** is the strip of artwork past the finished edge of the page, printed and then guillotined off, so that a millimetre of drift in the trimmer shows your background instead of a white sliver; KDP requires 0.125 inch of it on every outer edge of a cover. The script reads the page count out of the interior with `pdfinfo`, multiplies by a per-page paper thickness, and lays back, spine and front onto one canvas with that bleed on every side. On this book:

```
Pages: 105
spine: 0.265 in
wrote output/pdf/paperback-wrap.pdf
    (11.515000 x 8.750000 in, includes 0.125 in bleed)
```

(The script prints the first two on one line, separated by a pipe. They are split here so the line survives being quoted inside a Markdown table, which is where the mechanical review found it was fragile.)

Read that width: 5.5 plus 5.5 for the two covers, 0.265 for the spine, and 0.25 for bleed on the two outer edges. The height is 8.5 plus 0.25. The script emits the PDF with an exact media box rather than a pixel count, because KDP rejects covers whose dimensions round.

Then the script prints four instructions, and they are the most important output it produces:

```
1. OPEN KDP's cover calculator for your trim,
   paper, and 106 pages.
2. Compare its spine width to the 0.265 in above.
3. Confirm every spine letter sits inside the safe
   zone of KDP's template.
4. Rebuild if KDP's spine differs, then order a
   physical proof.
```

The per-page constant in the script is 0.0025 inch, which approximates cream 60 pound paper. White paper is thinner. Premium colour is different again. **KDP's own cover calculator is the authority, and the script is an approximation that tells you so.** This is the Class B rule from chapter 5 applied to geometry: a number owned by Amazon never gets asserted flatly by me, in a script or in a book.

The first thing you can only learn by uploading: Amazon does not allow text on the spine below a page-count threshold, because a thin spine cannot hold legible type. At 106 pages this book clears the line I found, barely. Check the current threshold on KDP's paperback formatting page before you design a spine you cannot use.

A note about the numbers in that output, and it is the reason they are printed rather than described. This chapter has quoted three different page counts. The first draft said 80 pages and a 0.200 inch spine, which is what the interior measured while three chapters did not yet exist. After the draft was finished it was 100 and 0.250. After the review pass added eleven paragraphs it was 106 and 0.265, which is what you see above, and what the cover in `output/pdf/` was built from.

Nothing was ever wrong with the arithmetic. The input moved underneath a number that had been written down as a fact, three times, in a book whose subject is that exact failure. Two consequences, and both are load-bearing. The cover target reads the page count out of the built PDF rather than out of a variable, so the wrap is never computed from a remembered number. And `make covers` is the last thing you run, after the last prose edit, because a spine is a fact about a finished manuscript and not about a nearly finished one.

Preflight, which refuses to claim too much

`make preflight` validates what is countable and stops:

```
PDF: 106 pages, 396 x 612 pts
EPUB: ZIP container and mimetype valid
epubcheck not installed; formal EPUB validation
```

```
    still required.

Automated preflight passed. Kindle Previewer and a
    physical proof are still required.
```

Four checks. The interior's page size in points against the expected trim, since 396 by 612 points is exactly 5.5 by 8.5 inches. Every font embedded, via `pdffonts`, because an unembedded font is a rejection. The EPUB's ZIP container and its mimetype. And EPUBCheck, if it is installed.

It was not installed on my machine, and the script says so in one line rather than skipping quietly or pretending the check passed. That is the whole design philosophy of this pipeline in four words: **check what you can verify.** A preflight that claimed EPUB validity without EPUBCheck would be worse than no preflight, because you would believe it.

The last line is not decoration either. Kindle Previewer renders your EPUB the way a Kindle will, which is the only way to catch a broken table or an image that overflows on a small screen. A physical proof catches gutter problems, ink bleed-through, and the discovery that your chosen font is unreadable at 11 point on cream paper.

The decision this chapter closes

Four files on disk, a passing preflight with its caveats read rather than skimmed, a spine number cross-checked against Amazon's calculator, and a warning count of zero on the interior build.

Next is the dashboard, which is the half of publishing that no Makefile can do for you.

Chapter 16: The KDP Dashboard

> Account and tax setup first. Title, subtitle, series, description. Categories and the seven keyword slots. ISBN and ASIN. Pricing and the two royalty models. The AI disclosure questions. Previewers, the proof copy, and publish.

The build was the easy half. This is the half where a book that took nine months gets described in 150 words, priced in a currency you did not expect, and asked three questions about AI.

Do the boring part first, on a different day than the exciting part: create the KDP account and complete the tax interview and the bank details. The tax interview is a form about withholding, it takes twenty minutes, and until it is done you cannot publish anything. Nobody wants to discover that at 11pm with a finished book.

Two products, one book

KDP treats the ebook and the paperback as separate products with linked listings. You create the ebook, upload the EPUB and the cover JPG, then create the paperback from the same title and upload the interior PDF and the wrap PDF. The metadata is entered twice, so keep it in one file and copy from there. `metadata.yaml` and `assets/covers/copy/back.txt` are that file in this pipeline, which is the point of writing the blurb into the repository rather than into a browser tab.

The fields that matter

Title and subtitle. Exactly as they appear on the cover, character for character. A mismatch between cover and metadata is a review-team rejection, and it costs a day.

Series and author name. Leave series empty unless a second book genuinely exists in the same one, because it is easier to add than to remove. Keep the author name identical across both products and across every future book, or your Author Central page splits in two.

Description. This is the back-cover promise from chapter 3, at 120 to 150 words, and no other text you write outside the book does as much work. It is not a summary. It is the first two minutes of the reading experience. Mine lives in `assets/covers/copy/back.txt` so the paperback back cover and the Amazon listing cannot drift apart. A description written in the browser at midnight will drift from your cover within a week.

KDP accepts limited HTML in the description. Use a couple of paragraph breaks and one bold line at most. A wall of text loses the reader on a phone, which is where most of them are.

Categories and keywords

You choose up to three categories and seven keyword slots, and both are search positioning rather than description.

For categories, the shelf test from chapter 3 does the work: the three books a browser would put yours next to tell you which categories those books are in, and you can look. Choose the most specific category that is honestly correct. A niche category where your book is genuinely relevant beats a broad one where it is invisible.

For keywords, seven slots, each a phrase rather than a word. Two rules that took me a book to learn. Do not repeat words that are already in your title or subtitle, because those are indexed anyway and you are wasting a slot. And write the phrase a reader would actually type, not the phrase you wish described your book. "write a book with ai" is a search. "AI-assisted authorial workflow" is not.

Concrete beats clever in all seven slots.

ISBN and ASIN

The ebook gets an ASIN, assigned by Amazon. You need no ISBN and should not buy one for it.

The paperback needs an ISBN, and KDP gives you one free. The free one is fine, with one consequence: it lists Amazon as the publisher of record, and it cannot be used anywhere else. If you plan to sell the same paperback through another printer or distributor, buy your own ISBN instead. Either way, the rule that matters: **one ISBN per binding, per edition, forever.** A paperback and a hardcover are two ISBNs. A second edition with new content is a new ISBN. Never reuse one, because libraries and retailers key on it, and a reused ISBN is a mess you cannot clean up later.

Pricing and the two royalty models

Ebooks have two royalty options, a higher rate inside a price window and a lower rate outside it, and the higher one has a delivery charge based on file size deducted per sale. Paperbacks pay a percentage of list price minus a printing cost that depends on page count, ink, and marketplace.

I am not printing the current rates, the current price window, or the current printing cost. They change, this is a book, and a stale royalty table is worse than no table. What you do instead, on the day you price:

1. Open KDP's printing-cost calculator, enter your trim, page count, and ink.
2. Note the printing cost. That is your floor: below it the paperback loses money per copy, and KDP will refuse the price anyway.
3. Check the current royalty bands on KDP's pricing page and see where your intended price sits.
4. Look at what the three books from your shelf test charge. That is your ceiling, and it is set by your readers, not by your costs.

For this book, a 106-page paperback, the printing cost is low and the practical constraint is the shelf, not the arithmetic. For a 400-page novel the arithmetic starts to bite, and it is the reason a lot

of self-published paperbacks are priced higher than their traditionally published neighbours.

The three AI questions

KDP asks, separately, whether the book contains AI-generated text, AI-generated images, and AI-generated translations. It distinguishes AI-generated content from AI-assisted content, where you created the work and used AI to refine it, and the distinction it draws is the one you must read on the form itself, because the wording has already changed at least once.

Answer truthfully. Two reasons, in the order that will actually motivate you.

The dull reason: your answers must be consistent with the copyright page of the book you just uploaded. This project's front matter carries a disclosure paragraph, so a "no" in the dashboard would contradict the book's own page 2, and that inconsistency is visible to anyone who looks.

The real reason: you have a review record. The bible, the six reports, the reconciled file, the response file, and a git history where each decision has a date and an author. You are not in a position where the honest answer costs you anything, and that is precisely what the pipeline in part four was for.

Previewers, proof, publish

Before you press publish:

- **Kindle Previewer** on the EPUB. Check the table of contents, one code block, and one table on a phone-sized screen.
- **KDP's Print Previewer** on the interior. Check the first page of a chapter, a code block near a page break, and the gutter.
- **Order a proof copy.** It costs printing plus shipping and takes a few days. Read the first chapter on paper. You will find two things: one typo that survived four review passes, and one layout problem no previewer showed you.

Then publish. KDP states its own review window on the confirmation screen in front of you, and that screen is the authority rather than this sentence.

The second thing you can only learn by uploading: the paperback and ebook listings appear separately, before Amazon links them. For about a day your book exists twice, unconnected, and both listings look wrong. Nothing is broken and there is nothing to fix, which is impossible to know in advance and unnerving the first time.

The decision this chapter closes

A live listing, a proof copy on the way, categories and keywords chosen from the shelf rather than from hope, and three AI answers that match the copyright page of the book itself.

Chapter 17: After Publish

> The first week. What a new edition costs for each format. Reviews as thin data. The one habit that makes book two faster. What to change before the next run.

The book is live. Two things are true at once: nothing will happen for a while, and you will check the dashboard nine times before lunch.

Close the tab. The sales report has a reporting delay, the numbers in week one are noise, and the only action available to you is one you should not take, which is rewriting the description every eleven hours.

New editions are cheap for one format and not the other

The ebook is genuinely easy to update. Fix a typo, rebuild the EPUB, upload it to the existing listing, and the corrected file replaces the old one within a day. Readers who already bought it may or may not get the update automatically, which is a reason not to treat this as a substitute for the gate in chapter 14, but a correction costs you ten minutes.

The paperback is not. Any interior change reflows pages, a page-count change changes the spine width, a new spine means regenerating the wrap, and a significantly different book needs a new ISBN. The realistic policy: batch paperback corrections, and only republish when you have enough of them to justify ordering another proof.

This is where the git history pays off in a way that is hard to appreciate before you need it. A second edition starts from a clean repository with the bible, every review report, and every response file intact. You are not reconstructing what you decided. You are reading it.

Reviews are data, and the sample is thin and biased

The first ten reviews are not a measurement of your book. They are a measurement of who found it early.

Read them for the vocabulary. When three readers use the same word for the same chapter, whether that word is thin, or dense, or exactly what I needed, that is a signal about your description as much as about the chapter, because the description set the expectation they are reporting against. Two of my second novel's early reviews complained about a thing the blurb had promised badly, and the fix was the blurb.

Do not answer reviews. Do not ask friends for reviews. Do not read the one-star review twice.

The habit that makes book two faster

At the end of book one you own something more useful than the book: a working method with your name on it. The voice guide is calibrated. The agent definitions have been through a real orbit and you know which two produce the findings you actually use. You know your own crutch words and your own tics, by name and by rate per thousand.

My third novel took six weeks against the first one's four months, with the same tools and the same model. All of the difference was in files that already existed.

So do this while it is fresh, in the week after publishing, and give it an hour:

- Update `bible/voice.md` with the tells you actually caught yourself using, and delete the ones that never fired.
- Update the agent definitions with the checks you ended up asking for by hand.
- Write down the one thing that cost you the most time and what would have prevented it. Mine, in order across four books: no outline, an unverified word count, and a font that silently dropped diagrams from a print interior.
- Delete what you did not use. A pipeline with three unused review dimensions develops a maintenance cost and a guilt tax.

What I am changing before the next run

Three things, concretely, so this chapter is a report rather than advice.

An early orbit on the outline and the first three chapters is now mandatory, not optional. On this book it would have caught the two dead chapters before they were in the outline at all.

The build gets a warning gate. Any `make print` that emits a warning fails the gate, because a successful build with 71 warnings produced a paperback with holes in it and I only found that by reading the log.

And every counting tool gets a known-answer test on the day it is written, not on the day a chapter needs to describe it.

The decision this chapter closes

The pipeline is reusable, the review record is archived with the book, and you have written down the single thing you will do differently. That is the end of the method.

Afterword: The Honest Ledger

> What the machine drafted, what the author decided, what the machine got wrong, what the orbit caught, what a human caught. Consistent with the copyright page and the KDP answers. No triumphalism, no apology.

This book was produced with the pipeline it describes, which means the ledger is available and printing it is the least I can do.

What AI drafted. Prose in all 17 chapters, from beats I wrote, against a voice guide extracted from my own published articles, one chapter per session, following the drafting prompt in chapter 8.

What I decided. The premise, the reader, the promise, the locked decisions, the outline and every end beat, the eight rules the book obeys, what to cut, and every disposition in the response file. Two chapters in the outline were killed by my own end-beat rule before a word of them existed.

What the machine got wrong here. Four things, all caught by verification rather than by reading, and two of them were numbers the book had written down about itself.

A stale self-referential number. Chapter 9 claimed the em-dash grep found one hit in the manuscript, in a code block. By the time later chapters printed the same command, that was no longer the count. A book about verification printing an unverified number about itself is the exact failure it warns against, and the correction is in the response file.

A stale build number. Chapter 15 printed the cover script's output as 80 pages with a 0.200 inch spine. True when the chapter was written, and wrong by the time the last three chapters existed. The interior went to 100 pages, then to 106 as the review rounds added paragraphs, and the spine with it. The arithmetic was never wrong. The input moved underneath a fact, twice, in the same book, which is why chapter 15 now explains why `make covers` reads the page count out of the built PDF rather than out of a variable.

Inherited boilerplate. The front matter carried "This is a work of fiction" for a while, because the template came from three novels and nobody had told it this book was not one.

The confident measurement. `make wordcount` was 90% right for three books, and its error was invisible because it always erred in the same direction. It took writing a chapter about it to test it.

What the review orbit caught that I would not have. The chapter ledger showed that chapters 13 and 14 read as one chapter split in two, which neither chapter can show you from the inside. The voice pass caught the stale em-dash number, the one blocking finding of the first orbit. The second orbit, run with six isolated agents instead of six passes in one context, caught nine blocking defects, three of which I had introduced myself while fixing the first orbit's findings.

What the orbit did not catch, and I should not credit it with. The missing box-drawing glyphs were mine. I found them in `make print`'s output while writing chapter 15, before any review pass

existed, which is what the mechanical report means by "fixed before this pass". Two of the six passes then miscounted the book's own structure. One of those wrong counts was hiding a real violation of the voice guide. Six of the twenty-three countable findings in this orbit did not survive being counted, and a seventh miscount was mine: the script I wrote to settle one of them passed its own self-test while measuring the wrong thing.

What I got wrong about my own refusals. The first pass over the reviews produced eight items I declined or accepted as facts about the book, each with a written reason. That is the discipline this book spends a chapter defending. Then I went back and applied them anyway. Seven of the eight were worth doing, and the eighth was a rule rather than a defect. The reasons had not been false; they had been estimates of cost, made without attempting the work, and a refusal that has never been attempted is an estimate wearing a decision's clothes. The rule in chapter 13 stands, with that amendment attached.

What no agent caught. That the orbit for this book ran as sequential passes in one session rather than as six independent blind subagents, which is weaker evidence than the method asks for. I know because I ran it. It is in the response file as a deviation, and it is the reason chapter 11 documents both ways of running the orbit instead of assuming you have subagents.

What is disclosed where. The copyright page discloses AI drafting and editorial review under my direction. The KDP dashboard answers say the same thing. This afterword is the detail behind both. The three statements agree because they were written from the same file, which is the habit the whole book is arguing for.

The uncomfortable question at the end of a project like this is whether the book is mine. My answer is procedural, and I prefer it to a philosophical one. The premise is mine. The voice guide came out of writing I published before any of this existed. Every structural decision has my name and a date on it in a git history, and every review finding was either applied by my decision or declined with my reason. I can produce all of it on request.

That is not a claim about how much of the typing I did. It is a claim about who was responsible, and responsibility is the part that a reader is actually buying.

The pipeline is in the repository. Take it, delete the parts you do not use, and write the thing you have been keeping in a folder called notes.